

# Ejercicio 17: Optimización de Paradas en Coche Eléctrico

## Enunciado

Debemos hacer un recorrido entre dos ciudades en un coche eléctrico, en el menor tiempo posible. El vehículo tiene una autonomía de  $x$  kilómetros después de recargarlo completamente (en las condiciones habituales de ese recorrido). El sistema de ayuda del vehículo dispone de la información sobre los cargadores, con las distancias entre los mismos a lo largo del recorrido. Teniendo en cuenta que el tiempo de carga no es cero, se desea hacer el menor número posible de paradas.

## 1. Modelado del Problema

### Definiciones

- **Puntos de Interés:** Sea  $C = \{c_0, c_1, \dots, c_n, c_{n+1}\}$  el conjunto de ubicaciones.
  - $c_0 = 0$  (Ciudad de origen).
  - $c_1, \dots, c_n$  (Ubicación de los cargadores intermedios).
  - $c_{n+1} = D$  (Ciudad de destino).
- **Distancias:**  $d[i]$  es la distancia acumulada desde el origen hasta el punto  $c_i$ .
- **Restricción:** Autonomía máxima del vehículo =  $x$ .
- **Conectividad:** Es posible ir del punto  $j$  al punto  $i$  ( $j < i$ ) si:

$$d[i] - d[j] \leq x$$

## 2. Estructura de Programación Dinámica

### Definición de la Función Óptima

Sea  $DP[i]$  el **mínimo número de recargas** necesarias para alcanzar el punto  $c_i$ .

### Relación de Recurrencia

Para llegar al punto  $i$  con el coste mínimo, buscamos un punto previo  $j$  que esté al alcance de la autonomía y que tuviera el menor número de paradas acumuladas:

$$DP[i] = \min_{j < i, d[i] - d[j] \leq x} \{DP[j] + 1\}$$

### Casos Base e Inicialización

- **Origen:**  $DP[0] = -1$ . (Nota: Se usa -1 porque al salir del origen la batería está llena y no cuenta como parada. El primer salto sumará +1 resultando en 0 paradas reales en los cargadores).
- **Inalcanzables:**  $DP[i] = \infty$  para todo  $i > 0$ .

### 3. Algoritmo y Complejidad

#### Pseudocódigo

```

Entrada: d[0..n+1] (distancias), x (autonomía)

DP[0] = -1
pred[0..n+1] = null

Para i desde 1 hasta n+1:
    Para j desde 0 hasta i-1:
        Si (d[i] - d[j] <= x):
            Si (DP[j] + 1 < DP[i]):
                DP[i] = DP[j] + 1
                pred[i] = j

Si DP[n+1] == infinito:
    Retornar "No existe camino posible"
Sino:
    Reconstruir camino usando pred[] (backtracking)
    Retornar DP[n+1]

```

#### Análisis de Coste

- **Temporal:**  $O(n^2)$  debido a la exploración de todos los posibles saltos previos para cada punto.
- **Espacial:**  $O(n)$  para los vectores de estado y reconstrucción.